协同仿真

NEMU 扩展解析

协同仿真在昆明湖 V2 RTL 上增加了特色的 NEMU 指令扩展，并使用 DiffTest 对照两者执行结果。

文件	修改
src/isa/riscv64/include/isa-all-instr.h	在向量指令表加入 vdot
src/isa/riscv64/instr/rvv/decode.h	在 OPIVV 表匹配实验 funct6，保留 vm 的执行检查
src/isa/riscv64/instr/rvv/vcompute.h	检查配置，逐元素做带符号乘法和 int32 累加，再完整写入目的寄存器
patches/nemu.patch	保存上述三项修改，应用于隔离的 work/NEMU

执行函数先调用 require_vector 检查向量状态和 vill，再检查 SEW、LMUL、vl、vstart 与 vm。所有 16 对输入读完后才写结果，因此 vd 与任一源重叠时仍使用原输入。低 64 位赋值为 (uint32_t)sum，自动将该字的高 32 位清零；高 64 位另写零，形成完整的高 96 位清零结果，即:

```
int32_t sum = 0;
for (int i = 0; i < 16; ++i) {
 const int32_t a = (int8_t)vreg_b(id_src1->reg, i);
 const int32_t b = (int8_t)vreg_b(id_src2->reg, i);
 sum += a * b;
}
vreg_l(id_dest->reg, 0) = (uint32_t)sum;
vreg_l(id_dest->reg, 1) = 0;
```

NEMU 用 C 标量循环计算结果，使用独立的 C 标量参考检查输出，vdot 的参考结果由 NEMU 计算。

NEUM 构建流程

香山与 NEMU 的默认 flash 启动代码不同。nemu-compat.patch 增加按 DiffTest 提供的实际字节装入与 DUT 相同的 flash，统一程序启动状态。

异常 mtval 将通用译码检测到向量状态关闭、vill、vstart 非零等情况时保留指令位；VIPU 的配置异常报告零。NEMU 从自身指令字段和 CSR 判断对应情况，mcause、mepc、mtval 与架构寄存器继续由原有 DiffTest 比较。

在作品根目录执行。首次运行或修改 RTL 后先构建模拟器；随后构建 NEMU 和测试程序。已有匹配构建时可直接使用后续命令。

```
# 在克隆后的作品仓库根目录执行
source scripts/env.sh
bash scripts/build_emu.sh
bash scripts/build_nemu.sh
python3 scripts/test_nemu.py
bash scripts/build_app.sh vdot
bash scripts/build_app.sh illegal
bash scripts/build_app.sh vadd
KKS_TIMEOUT=3600 bash scripts/run_emu.sh vdot custom diff
KKS_TIMEOUT=1800 bash scripts/run_emu.sh illegal custom diff
bash scripts/run_emu.sh vadd custom diff
```

custom diff 选择作品目录的新模拟器和 NEMU 参考库。一次执行的完整原始日志写入 build/logs，scripts/stage3-run.sh 可依次重建第三阶段程序、执行全部测试并生成波形数据与图片。

NEMU 指令测试结果与波形对比

scripts/test_nemu.py 通过真实 DiffTest 共享库接口执行 tests/nemu/loop.S。每条指令执行后读取完成标志，收到本组结果后停止，再开始下一组。133 组输入包括极值、零、只激活上八对和固定种子随机输入，每组检查五种寄存器变体的完整 128 位结果，共 665 次比较。vcsr 轮换覆盖八种状态。

八个非法配置分别在独立进程中执行。异常处理程序记录 mcause、mepc，恢复向量访问后保存原 v10；结果检查 cause=2、异常 PC 对应 dot 编码、完整旧值。

以下为协同仿真结果：

```
NEMU vectors PASS vectors=133 result128=checked vcsr=preserved
NEMU vl15 PASS vectors=1 result128=checked vcsr=preserved
NEMU sew16 PASS vectors=1 result128=checked vcsr=preserved
NEMU lmul2 PASS vectors=1 result128=checked vcsr=preserved
NEMU vstart1 PASS vectors=1 result128=checked vcsr=preserved
NEMU masked PASS vectors=1 result128=checked vcsr=preserved
NEMU vl0 PASS vectors=1 result128=checked vcsr=preserved
NEMU vill PASS vectors=1 result128=checked vcsr=preserved
NEMU vs_off PASS vectors=1 result128=checked vcsr=preserved
NEMU SUITE PASS vector_cases=133 aliases=5 full128_checks=665 illegal_modes=8 vcsr_states=8


PIPELINE PASS dependent_dots=33 interleave=PASS branch_skip=PASS
VDOT PASS vectors=216 aliases=5 full128_checks=1080 vcsr_states=8
HIT GOOD TRAP at pc = 0x80000a3c
KKS_RUN_EXIT code=0

ILLEGAL PASS modes=8
HIT GOOD TRAP at pc = 0x80000326
KKS_RUN_EXIT code=0
```

程序	实际检查	结果
vdot	216 组输入、五种寄存器变体、1080 次完整结果比较；连续依赖、标准 VIPU 交错与分支跳过	PASS，GOOD TRAP，退出码 0
illegal	八种非法配置、异常 PC、cause=2、旧目的寄存器保持	PASS，GOOD TRAP，退出码 0
vadd	16 个标准向量加法元素及字节回绕	PASS，GOOD TRAP，退出码 0
bench	RVV 基线与 vdot 的五种极值等价性、15 组计时样本的完整结果	PASS，GOOD TRAP，退出码 0
smoke	同一输入的 RVV 基线与 vdot；上八对非零、低八对为零	PASS，GOOD TRAP，退出码 0

波形程序顺序执行标准 RVV 和 vdot，输入低八对均为零，高八对非零。两种计算结果均为-32416。PC 和符号地址从本次 smoke.elf 读取，执行输入与输出使用相同 ROB 。

vdot 观察点	sim_timer	定位
最后一次译码观察	5012	PC=0x80000152，指令字 0x0e848557

vdot 观察点	sim_timer	定位
ROB 分配	5016	ROB=(0,105)
VIPU 输入握手	5020	完整两个 128 位源，fuOpType=254
VIPU 输出握手	5022	同一 ROB，结果为 0x0000000000000000000000000ffff8160
ROB 提交	5024	同一 PC 的提交

同一 PC 此前还出现在译码端，但没有进入目标 ROB。表中列出该 ROB 分配前最后的译码结果，保留的波形包含全部选取信号及变化。

记录位置与处理过程

最终原始日志保存在 stages/03-verification/logs；波形及事件数据保存在 waves，图片保存在 images。NEMU 构建.log 含实际参考库信息，点积协同仿真.log 与非法配置协同仿真.log 保留各组输出及退出码。

首次构建时，defconfig 未启用浮点选项，SoftFloat 没有被自动下载。构建脚本先获取锁定的依赖提交，再执行 defconfig 和编译。

首次几项仿真已输出 PASS 和 GOOD TRAP，但运行期间修改脚本导致 Bash 收尾时读取错位，固定后重新执行，最终五个整核程序均得到退出码 0。

GTKWave 已实际打开精简波形，并自动选中 17 个信号。